

Project Proposal

Campus Parking Availability and Violation Monitoring System

Sharon Zhu, Student ID: 1666262

1. Introduction

The University of Waikato does not have a system that shows students where to park. Students must drive around and look for a space by themselves. There is no way to check parking space before arriving. There is also no easy way to see parking violations, such as a car in a reserved park, or a car parked without payment during paid hours.

The University of Waikato parking page lists real gate areas for students: Gate 1, Gate 2b, Gate 3A, Gate 3B, and Gate 10. Paid hours are 8:30am to 4:30pm, Monday to Friday. Parking is free outside these hours. Numbered or named parks are reserved for permit holders at all times. Parking there can lead to wheel-clamping. There are also mobility parks. These are free at all times for drivers with a valid mobility card. This project uses these real gate names and real rules, instead of made-up parking zones.

This project will build a web system to help with this problem. The campus does not have working sensors or cameras to detect parking spaces. Because of this, the system will use a backend program that creates parking events on its own. This program runs all the time in the background. It represents the kind of data that sensors would normally send. Users do not need to do anything. They just open the page and see live occupancy for each gate area. A rule engine checks each event against the real parking rules. It flags violations such as a car in a reserved park, an unpaid car during paid hours, or a car in a mobility park without a valid card. A priority list helps patrol staff choose which gate area to check first.

I would prefer to use real occupancy data. I have contacted the parking, security, and IT teams to ask about this. In consultation with my lecturer, I will use my own virtual data if real data is not available. My project plan does not depend on getting a reply. The system works fully with virtual data from the start. If real data arrives later, I will use it to check and improve my results.

I chose this project because it is a real problem on campus. I also wanted to use skills from last trimester, such as web development, databases, and problem solving, in one project.

2. Objectives

- Show real-time occupancy for each student gate area (Gate 1, Gate 2b, Gate 3A,

Gate 3B, Gate 10), at gate level, not single car park level.

- Create realistic parking events in the backend, since the campus has no working sensors. Users do not need to check in or check out.
- Detect common violations automatically, based on real University of Waikato parking rules.
- Predict how full a gate area will be at a future time, using past data.
- Compare two prediction methods and discuss which one is more accurate.
- Flag cases where real occupancy does not match the prediction. This may point to unregistered parking.
- Give patrol staff a priority list of gate areas to check, based on violation history and current status.
- Support two user roles: normal user and admin, with different access rights.
- Make the system work well on both desktop and mobile browsers, using one responsive design.
- Provide a simple kiosk display page that could be shown on a screen at a car park entrance.

3. Requirements

Functional Requirements

- Users can register and log in with a username and password.
- A backend program creates parking events (plate number, gate area, park type, payment status) on its own, on a set schedule.
- The system checks that each generated plate number has a valid format, using a regular expression.
- Users can view current occupancy for all gate areas in real time, with no action needed from them.
- The gate occupancy page refreshes itself every few seconds, so the numbers stay current without a manual page reload.
- The rule engine marks an event as a violation if the car is in a reserved numbered park.
- The rule engine marks an event as a violation if the car is parked during paid hours (8:30am-4:30pm, Monday to Friday) and payment is marked as unpaid.
- The rule engine marks an event as a violation if the car is in a mobility park with no valid mobility card on record.
- Motorbike parks, and hours outside 8:30am-4:30pm on weekdays, are treated as free. No payment violation applies.
- Admin can view all parking events and violation records.
- Admin can set rules, such as paid hours and which gate areas are reserved.

- The system predicts gate occupancy for a future time slot, using two methods: moving average and simple linear regression.
- The system compares the two prediction methods and shows which one was closer to the real result.
- The system compares predicted occupancy to actual occupancy and flags a large gap as unusual.
- The system generates a patrol priority score and order for each gate area.
- A kiosk display page shows large, simple text for each gate area, suitable for an entrance screen.

Non-Functional Requirements

- Pages should load in under 2 seconds under normal use.
- Passwords must be stored using hashing, not plain text.
- The layout should adjust automatically for desktop and mobile screen sizes, using one shared codebase.
- Code should be organised into small functions, with clear comments.
- The system should handle bad input, such as a wrong plate format, without crashing.
- A normal user account must not be able to reach admin-only actions, even by calling the API directly.

4. Functionality

In simple terms, the system will let a person do the following:

- Log in as a student, staff member, or admin.
- Open the page and see live occupancy for each gate area (Gate 1, Gate 2b, Gate 3A, Gate 3B, Gate 10), with no need to check in or out.
- See a list of gate areas with how full each one is, updating on its own every few seconds.
- See violations that the system has flagged, based on real rules: a reserved park, an unpaid car during paid hours, or a mobility park with no valid card.
- As an admin, see a list of violations and choose how to handle each one.
- As an admin, see a suggested order of gate areas to patrol, based on risk.
- As an admin, see a chart comparing predicted and actual occupancy, for two prediction methods.
- Open a kiosk display page with large text, made for a screen at a car park entrance.
- Use the same website on a phone browser, with a layout that adjusts to the smaller screen.

5. Tech Stack

- Frontend: React, with a responsive layout for both desktop and mobile browsers.
- Backend: Node.js with Express, used to build REST API endpoints.
- Database: MySQL, using Sequelize (or mysql2) for schema design and queries.
- Authentication: JWT (JSON Web Token) for login, with role-based access control for normal users and admins.
- Data generator: a Node.js script that runs on a schedule (using node-cron or a similar tool), creating parking events with realistic busy and quiet periods.
- Live updates: the frontend polls the API every few seconds using setInterval, so the page content updates without a full page reload.
- Core logic: JavaScript functions for the rule engine, a weighted score for patrol priority, and two prediction methods (moving average and linear regression).
- Text processing: regular expressions for plate number checks.
- Testing: Jest for unit tests, Postman for API and security tests.
- Scripting: shell scripts to set up the database and seed starting data.
- Version control: Git and GitHub.

6. Timeline

The plan below follows the trimester teaching schedule, including the recess weeks.

Week	Focus
1	Confirm topic. Finish proposal. Set up GitHub repo and project structure.
2	Design MySQL tables: users, gate areas, parking events, violations.
3	Build basic Express API: login, register, gate list, get occupancy.
4	Build gate occupancy logic and the violation rule engine.
5	Build the backend data generator. It creates parking events automatically, with busy and quiet times.
6 -7	Teaching recess. Read about moving average, linear regression, and priority scoring.
8	Build the React frontend. Add login and the gate status page. Make it responsive for mobile browsers.
9	Build the admin dashboard: violation list and rule settings.
10	Build occupancy prediction. Add a second prediction method and compare accuracy.
11	Build the patrol priority score and the kiosk display page with auto-refresh.
12	Write unit tests and API tests. Add security tests for role-based access.
13	Fix bugs found in testing. Polish the responsive layout.
14	Final polishing. Write the report. Prepare the presentation.

The patrol priority score and kiosk page (Week 11) are the parts I may reduce in scope if earlier weeks take longer than planned. The core features, such as the data generator, gate occupancy, and violation rules, will be finished first. The rest of the system depends on these.

7. Deployment Plan

During development, the system will run on my own computer. The backend and MySQL database will run locally. For the final presentation, I plan to put a working demo online. I will use a free hosting option, such as Render or Railway, for the backend. I will use a free MySQL hosting option, such as Railway or Aiven, for the database. The frontend can be hosted on a static hosting service, such as Vercel or Netlify.

This is a course project, not a live campus system. Full production deployment, such as a paid host or a real domain name, is not required. The goal is to have a working demo that I can show and test during the presentation.

8. Testing Plan

Testing will happen at three levels: unit testing, integration testing, and security testing.

Unit Testing

- Test the violation rule engine with sample events: a reserved park, an unpaid car during paid hours, and a normal valid case.
- Test both prediction methods with sample historical data, and check that the results are reasonable.
- Compare the two prediction methods on the same data set and record which one is more accurate.
- Test the anomaly check by adding a known unusual value and checking that it gets flagged.
- Test the patrol priority score with sample gate areas that have different violation history and occupancy.

API Testing

- Use Postman to test each API endpoint, such as login, get gate occupancy, get violations, and admin rule settings.
- Test with valid input and also with bad input, such as a wrong plate format or missing fields.

Security Testing

- Try to call an admin-only endpoint using a normal user's login token, and check the request is refused.
- Try to call a protected endpoint with no login token at all, and check the request is refused.
- Check that an admin's login token is correctly accepted for admin-only endpoints.

I will also keep a short bug log during development. It will list bugs found, how they were fixed, and any bugs still open at the end of the project. This will be included in the final report.

9. References

University of Waikato. (n.d.). Parking. University of Waikato. Retrieved from the University of Waikato website, Hamilton campus parking information page.

10. Conclusion

This project will build a system to show parking occupancy on campus, detect common violations, and help patrol staff work more efficiently. It uses a priority list and simple prediction to support this. The design matches how the campus actually manages parking today, using gate areas and staff patrols instead of car park sensors.

I have used the real parking rules and gate names from the University of Waikato parking page as the basis for this project. I have also contacted the parking, security, and IT teams to ask about real occupancy data, since the number of car parks per gate is not listed on the public page. At the time of writing, I have not received a reply.

In consultation with my lecturer, virtual data is an acceptable option if real data is not available. The project does not depend on a reply from these teams. The system runs fully on virtual data from a backend generator. It uses an adjustable number of car parks per gate. If real data arrives later in the trimester, I will use it to check and improve the prediction feature. I will note this update in a future progress report.

By the end of the trimester, I hope to have a working system that shows what a useful, realistic campus parking tool could look like, built with the skills I have learned in this programme.